FRONTEND ROADMAP

DAY 5

The Event Loop & Async JS

JavaScript does one thing at a time — so how does it wait for timers, clicks and data without freezing? Meet the event loop, promises and async/await.

WHAT WE COVER

- Sync vs async
- setTimeout
- The event loop
- Promises
- async / await

So far your code has run top-to-bottom, line by line. But real apps have to WAIT — for a timer, a button, a server. Today you'll learn how JavaScript stays responsive while it waits, and how to write clean code that runs 'later'. We'll learn it the best way: by predicting the output of small examples.

The Event Loop & Async JS

JavaScript is **single-threaded** — it runs one line at a time. The magic is how it handles things that take time without blocking everything else. Read each example, *guess the output first*, then check. That prediction game is the whole lesson.

ON THIS HANDOUT

01 Sync vs async: one thing at a time

Synchronous code runs in order, top to bottom — each line finishes before the next starts. Most code you've written so far is synchronous.

```js
console.log("1");
console.log("2");
console.log("3");
// Output:  1  2  3   (always, in order)
```

Asynchronous code starts something now but finishes **later** — a timer, a network request, a click. JavaScript doesn't stand still waiting; it moves on.

- Sync = “do this, then this, then this.”
- Async = “start this, keep going, and come back to it when it's ready.”

02 The problem: blocking code

JavaScript runs on a **single thread**. If one line takes a long time, **everything** freezes — the page can't scroll, buttons don't click. That's **blocking**.

```js
// Imagine this took 5 seconds to run:
const data = downloadHugeFileSync();  // <- page is FROZEN here
console.log("done");                  // nothing happens until it finishes
```

The fix: do the slow work **asynchronously** so JavaScript can keep responding while it waits. The rest of this handout is about how.

03 setTimeout & the callback pattern

`setTimeout(fn, ms)` says: "run this function **later**, after at least this many milliseconds." The function you pass in is called a **callback**.

```js
console.log("Start");

setTimeout(() => {
  console.log("Inside timeout");
}, 2000);              // run after ~2 seconds

console.log("End");
```

Predict the output. Many people say Start → Inside timeout → End. It's actually:

```
Start
End
Inside timeout        // runs LAST, even with 0ms!
```

- JS registers the timer and **keeps going** — it does not wait.
- The callback runs later, after the current code finishes. Even `setTimeout(fn, 0)` runs after the last line.

04 The event loop (the mental model)

Here's the whole idea in three parts. Keep this picture in your head:

- **Call stack** — where your code runs right now, one thing at a time.
- **Web APIs** — the browser handles timers, network, clicks in the background.
- **Callback queue** — finished async tasks wait here for their turn.

The **event loop** has one job: *when the call stack is empty, take the next waiting callback and run it.*

```js
console.log("A");

setTimeout(() => console.log("B"), 0);   // goes to the queue

console.log("C");

// Call stack runs A, C first (that's your code).
// Only when the stack is EMPTY does the loop pull B in.
// Output:  A  C  B
```

That's why **a 0ms timeout still runs last** — it has to wait for your current code to finish and the stack to clear.

05 Callbacks get messy: “callback hell”

Callbacks work, but when one async step depends on the next, they nest deeper and deeper into a “pyramid” that's hard to read and fix.

```js
getUser(1, (user) => {
  getPosts(user, (posts) => {
    getComments(posts[0], (comments) => {
      console.log(comments);   // finally!
      // ...and it keeps nesting -->
    });
  });
});
```

This is called **callback hell**. Promises were invented to flatten it out.

06 Promises: .then() and .catch()

A **Promise** is an object for a value that isn't ready yet. Think of a restaurant buzzer: you get it now, and it goes off **later** when your food is ready.

- **pending** — still working
- **fulfilled** — succeeded, gives you a value (handled by `.then()`)
- **rejected** — failed, gives you an error (handled by `.catch()`)

```js
const order = new Promise((resolve, reject) => {
  const foodReady = true;
  if (foodReady) resolve("Pizza is ready!");
  else           reject("Kitchen closed");
});

order
  .then((msg)  => console.log(msg))    // "Pizza is ready!"
  .catch((err) => console.log(err));   // runs only on reject
```

The big win: chaining. Each `.then()` passes its result to the next — a flat line instead of a nested pyramid.

```js
getUser(1)
  .then((user)  => getPosts(user))
  .then((posts) => getComments(posts[0]))
  .then((comments) => console.log(comments))
  .catch((err) => console.log("Something failed:", err));
```

07 async / await: clean async code

`async / await` is modern syntax that lets you write async code that **reads like normal, top-to-bottom code**. It's built on promises.

- Put `async` before a function to allow `await` inside it.
- `await` pauses *inside that function* until the promise settles, then gives you the value.
- Use a `try / catch` block to handle errors.

```js
async function showComments() {
  try {
    const user     = await getUser(1);
    const posts    = await getPosts(user);
    const comments = await getComments(posts[0]);
    console.log(comments);
  } catch (err) {
    console.log("Something failed:", err);
  }
}

showComments();
```

Same logic as the promise chain above — but it reads like the synchronous version. This is the style you'll use most.

08 fetch: talking to a server

`fetch(url)` asks a server for data and returns a **promise**. This is the real reason async matters — the network is slow and unpredictable.

```js
async function loadUser() {
  const res  = await fetch("https://api.example.com/user/1");
  const user = await res.json();   // .json() also returns a promise
  console.log(user.name);
}

loadUser();
```

The same thing written with `.then()`, so you recognise both styles in the wild:

```js
fetch("https://api.example.com/user/1")
  .then((res)  => res.json())
  .then((user) => console.log(user.name))
  .catch((err) => console.log("Request failed:", err));
```

Putting it together: predict the output

One last prediction game that ties the whole day together. Guess the order before you read on.

```js
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

console.log("4");
```

Output: 1, 4, 3, 2. Your normal code (1 and 4) runs first. Then promises are handled before timers, so 3 comes before 2. Don't worry about memorising that order — just remember: **sync code first, async callbacks after.**

> **QUICK RECAP**
>
> JS runs **one line at a time** · slow work goes **async** so nothing freezes · the **event loop** runs waiting callbacks once your code finishes · **promises** and **async/await** are the clean way to handle 'later'.